

Фінальна практична робота з SQL

Дата-аналітичне розслідування: «Куди подівся виторг?»

Ви — **молодший дата-аналітик** інтернет-магазину електроніки **TechMarket**.
Сьогодні вранці фінансовий директор написав вам:

► ЛИСТ ВІД СФО

«За підсумками січня виторг різко просів, хоча замовлень ніби не поменшало. Розберіться, що сталося, скільки грошей ми недоотримали і хто наші ключові клієнти. До кінця дня чекаю короткий звіт із цифрами.»

Ваше завдання — провести розслідування **мовою SQL**: крок за кроком звужувати пошук, поки не знайдете причину провалу й не порахуєте втрати. Наприкінці ви заповните **підсумковий звіт** (передостання сторінка) конкретними числами та іменами.

Тривалість	≈ 50 хвилин (Етапи 0–7). Етапи 8–9 — бонус, якщо лишиться час.
Що здати	Один .sql-файл із запитами (по одному на кожне завдання, з коментарем-номером) + заповнений підсумковий звіт.
Інструменти	PostgreSQL + будь-яке середовище: psql, DBeaver, pgAdmin або онлайн-пісочниця.
Правила	Можна користуватись офіційною документацією PostgreSQL. Не можна — готовими розв'язаннями.

Крок 0. Підготовка бази даних

1) Створіть порожню базу та 2) виконайте доданий скрипт **setup.sql** — він створить таблиці й заповнить їх даними:

```
# у терміналі
createdb techmarket
psql -d techmarket -f setup.sql

# або у psql:
CREATE DATABASE techmarket;
\c techmarket
\i setup.sql
```

Якщо працюєте у DBeaver/pgAdmin — просто відкрийте **setup.sql** і виконайте весь скрипт. Повний DDL схеми наведено також у **Додатку А**.

Схема бази (9 таблиць)

Таблиця	Призначення
categories	категорії товарів; самозв'язок parent_category_id для підкатегорій
suppliers	постачальники товарів (країна, рейтинг)
customers	клієнти магазину (місто, країна, активність)
employees	працівники; самозв'язок manager_id (ієрархія підпорядкування)

products	товари (ціна price, собівартість cost, залишок на складі)
orders	замовлення; employee_id = NULL означає онлайн-замовлення без менеджера
order_items	позиції замовлень (кількість, ціна, знижка)
payments	оплати замовлень (не кожне замовлення оплачене)
reviews	відгуки клієнтів на товари (оцінка 1..5)

► **УВАГА**

Домовимось про терміни одразу — використовуйте їх в усіх етапах:

- **Виторг** = сума по позиціях: $\text{quantity} \cdot \text{unit_price} \cdot (1 - \text{discount})$ для замовлень, статус яких **НЕ** 'cancelled'.
- **Статуси замовлень**: new, paid, shipped, delivered, cancelled.
- **Успішна оплата** = рядок у payments зі статусом 'success'.

ЕТАП 0 · Підготовка та знайомство з даними

≈ 4 хв

► ЦІЛЬ

Переконалися, що БД працює, і скласти перше уявлення про обсяг і період даних.

Зразок-патерн (можна виконати як є, щоб перевірити підключення):

```
SELECT count(*) AS customers FROM customers;
```

Завдання:

- 0.1** Порахуйте кількість рядків у кожній з 9 таблиць.
- 0.2** Виведіть 10 найновіших замовлень (сортування за order_date).
- 0.3** Які унікальні статуси замовлень присутні в базі?
- 0.4** Який діапазон дат замовлень (найраніша та найпізніша order_date)?

► ПІДКАЗКИ

count(*), ORDER BY ... DESC, LIMIT, DISTINCT, MIN()/MAX().

► ОЧІКУВАНИЙ РЕЗУЛЬТАТ

Ви знаєте обсяг даних і що аналізуєте період у **6 місяців** (серпень 2024 – січень 2025).

ЕТАП 1 · Базові метрики бізнесу

≈ 5 хв

► ЦІЛЬ

Зробити «знімок» здоров'я магазину одним поглядом — щоб було з чим порівнювати.

Завдання:

- 1.1** Загальна кількість замовлень і кількість унікальних клієнтів, що робили замовлення.
- 1.2** Загальний виторг за весь період (за формулою вище — не забудьте про cancelled і discount).
- 1.3** Середній чек (AOV) = виторг ÷ кількість завершених замовлень (підзапит або CTE).
- 1.4** ТОП-5 товарів за виторгом.

► ПІДКАЗКИ

```
SUM(quantity*unit_price*(1-discount)), JOIN order_items ↔ orders, WHERE status <> 'cancelled', GROUP BY, ORDER BY ... DESC LIMIT 5.
```

► ОЧІКУВАНИЙ РЕЗУЛЬТАТ

Один рядок зведених метрик і таблиця топ-товарів. Запам'ятайте загальний виторг — у Етапі 7 рахуватимете частки.

ЕТАП 2 · Динаміка виторгу по місяцях

≈ 6 хв

► ЦІЛЬ

Знайти, КОЛИ почалися проблеми.

Завдання:

- 2.1 Виторг по місяцях (групування за роком-місяцем `order_date`).
- 2.2 Поряд виведіть кількість усіх замовлень по місяцях і окремо кількість скасованих.
- 2.3 Подивіться на таблицю: у якому місяці виторг різко випадає? Чи зменшилась при цьому кількість замовлень?

► ПІДКАЗКИ

`date_trunc('month', order_date)` або `to_char(order_date, 'YYYY-MM')`; `GROUP BY 1`; зручний прийом PostgreSQL — `COUNT(*) FILTER (WHERE status='cancelled')`.

► ОЧІКУВАНИЙ РЕЗУЛЬТАТ

Помісячна таблиця, де видно «провал» одного місяця за виторгом при майже незмінній кількості замовлень. **Запишіть цей місяць** — далі копаємо саме в нього.

ЕТАП 3 · Наскільки саме впало? Віконні функції

≈ 6 хв

► ЦІЛЬ

Оцінити падіння в числах: місяць-до-місяця (MoM) та наростаючим підсумком.

Завдання:

- 3.1 Оформіть помісячний виторг як CTE і додайте стовпець «виторг попереднього місяця» через LAG.
- 3.2 Порахуйте абсолютну і відсоткову зміну до попереднього місяця (MoM, %).
- 3.3 Додайте наростаючий підсумок виторгу від початку періоду (`SUM()` OVER (`ORDER BY month`)).
- 3.4 Який місяць має найгірший MoM-%? Зафіксуйте конкретну цифру падіння.

► ПІДКАЗКИ

`WITH monthly AS (...), LAG(revenue) OVER (ORDER BY month), ROUND(..., 1), SUM(revenue) OVER (ORDER BY month).`

► ОЧІКУВАНИЙ РЕЗУЛЬТАТ

Таблиця з колонками `month`, `revenue`, `prev_revenue`, `mom_pct`, `running_total`. У вас є конкретний % падіння у проблемному місяці — він піде у звіт.

ЕТАП 4 · Чому впало? Розтин скасувань

≈ 7 хв

► ЦІЛЬ

Знайти джерело проблеми, зосередившись на проблемному місяці з Етапів 2–3.

Завдання:

- 4.1 Скільки замовлень скасовано в проблемному місяці і яка це частка від усіх замовлень місяця?
- 4.2 Розкладіть скасовані позиції цього місяця за **категоріями** товарів (`order_items` → `products` → `categories`). Яка категорія лідирує?

- 4.3** Розкладіть скасовані позиції за **постачальниками** (... → suppliers). Хто головний «винуватець»?
- 4.4** Через HAVING залиште лише постачальників, у яких у проблемному місяці скасовано більш ніж 3 позиції.

► **ПІДКАЗКИ**

Ланцюжок JOIN order_items → orders → products → suppliers/categories; WHERE status='cancelled' AND (фільтр місяця); GROUP BY, HAVING COUNT(*) > 3.

► **ОЧІКУВАНИЙ РЕЗУЛЬТАТ**

Конкретні **постачальник** і **категорія**, на які припадає основна маса скасування. Це ваша головна гіпотеза.

ЕТАП 5 · Підтвердження якості: відгуки

≈ 6 хв

► **ЦІЛЬ**

Перевірити гіпотезу незалежним джерелом — оцінками клієнтів.

Завдання:

- 5.1** Середній рейтинг (AVG(rating)) по постачальниках (reviews → products → suppliers). Як виглядає підозрюваний на тлі інших?
- 5.2** ТОП-5 товарів з найнижчим середнім рейтингом (мінімум 2 відгуки — HAVING COUNT(*) >= 2).
- 5.3** Через CASE розподіліть відгуки на негативні (1-2), нейтральні (3), позитивні (4-5) і порахуйте по місяцях. Чи зростає негатив перед проблемним місяцем?

► **ПІДКАЗКИ**

AVG(rating), ROUND, CASE WHEN rating <= 2 THEN ... END, GROUP BY.

► **ОЧІКУВАНИЙ РЕЗУЛЬТАТ**

Підтвердження, що у «винуватця» аномально низькі оцінки, а негатив накопичувався напередодні провалу.

ЕТАП 6 · Скільки грошей «протекло»? Витік виторгу

≈ 6 хв

► **ЦІЛЬ**

Порахувати недоотримані гроші — другу частину проблеми.

Завдання:

- 6.1** Знайдіть замовлення зі статусом shipped або delivered, у яких НЕМАЄ успішної оплати.
- 6.2** Скільки таких замовлень і на яку суму (за формулою виторгу)? Це і є «витік».
- 6.3** Застосуйте COALESCE, щоб замість відсутньої суми оплати показати 0.
- 6.4** У якому місяці витоку найбільше?

► ПІДКАЗКИ

`LEFT JOIN payments p ON p.order_id=o.order_id AND p.status='success' + WHERE p.payment_id IS NULL; або WHERE NOT EXISTS (SELECT 1 FROM payments ...); COALESCE(sum, 0).`

► ОЧІКУВАНИЙ РЕЗУЛЬТАТ

Кількість «неоплачених» відвантажень і сумарна сума витоку (грн) — теж піде у звіт.

ЕТАП 7 · Хто наші ключові клієнти?

≈ 6 хв

► ЦІЛЬ

Відповісти на другу половину запиту директора: де концентрація виторгу.

Завдання:

- 7.1** Чистий виторг по кожному клієнту (тільки замовлення з успішною оплатою). Виведіть ім'я та email; для клієнта без email покажіть «—» через COALESCE.
- 7.2** Проранжуйте клієнтів за виторгом (`RANK()` / `ROW_NUMBER()`). Виведіть ТОП-5.
- 7.3** Порахуйте частку (%) кожного клієнта в загальному виторзі через `SUM()` OVER (). Яку частку дають топ-3 разом?
- 7.4** (за бажанням) Перевірте «правило Парето»: скільки клієнтів дають 80% виторгу?

► ПІДКАЗКИ

`JOIN customers ↔ orders ↔ payments(status='success'); RANK() OVER (ORDER BY revenue DESC); revenue * 100.0 / SUM(revenue) OVER ()`.

► ОЧІКУВАНИЙ РЕЗУЛЬТАТ

Список ключових клієнтів і ступінь концентрації виторгу (частка топ-3).

Бонусні етапи (8-9) — якщо лишився час. Закривають оптимізацію та транзакції.

ЕТАП 8 · Бонус: оптимізація запиту

≈ 5 хв

► ЦІЛЬ

Подивитись «під капот» і пришвидшити запит.

Завдання:

- 8.1** Додайте перед запитом з Етапу 2 `EXPLAIN ANALYZE`. Знайдіть найдорожчий вузол плану (Seq Scan? Sort?).
- 8.2** Створіть індекс, що допоможе фільтрації/групуванню (напр., `orders(order_date)` та/або `order_items(order_id)`).
- 8.3** Повторіть `EXPLAIN ANALYZE`. Чи змінився план і час? Поясніть результат (на малих даних індекс може й не спрацювати — чому?).

► ПІДКАЗКИ

`EXPLAIN ANALYZE <запит>; CREATE INDEX idx_orders_date ON orders(order_date);`

► ОЧІКУВАНИЙ РЕЗУЛЬТАТ

Коротке порівняння планів «до/після» і висновок про роль індексів та розміру даних.

ЕТАП 9 · Бонус: виправлення даних у транзакції

≈ 5 хв

► ЦІЛЬ

Безпечно навести лад, не зіпсувавши базу.

Завдання:

- 9.1** Відкрийте транзакцію (BEGIN). Поставте «завислим» замовленням (shipped/delivered без успішної оплати) статус 'cancelled' через UPDATE.
- 9.2** Зробіть SAVEPOINT; виконайте ще один UPDATE (напр., товарам NordParts stock_quantity = 0), потім ROLLBACK TO SAVEPOINT — скасуйте лише його.
- 9.3** Перевірте проміжний стан SELECT-ом і зробіть ROLLBACK (нічого не зберігаємо — це вправа). Який рівень ізоляції захистив би іншу сесію від «брудного читання» цих змін?

► ПІДКАЗКИ

```
BEGIN; UPDATE ...; SAVEPOINT sp1; ...; ROLLBACK TO SAVEPOINT sp1; ROLLBACK;
```

► ОЧІКУВАНИЙ РЕЗУЛЬТАТ

Практичне розуміння атомарності, SAVEPOINT і рівнів ізоляції (READ COMMITTED vs READ UNCOMMITTED).

Підсумковий звіт аналітика

Заповніть на основі своїх запитів і здайте разом із .sql-файлом.

Аналізований період: _____ - _____
Проблемний місяць: _____
Падіння виторгу МоМ: _____ % (з _____ грн до _____ грн)
К-сть замовлень у проблемному місяці vs попередній: _____ vs _____ (чи впала? _____)
Головна причина — постачальник: _____ / категорія: _____
Підтвердження з відгуків (середній рейтинг винуватця): _____
Витік виторгу: _____ замовлень на суму _____ грн
ТОП-3 клієнти: 1) _____ 2) _____ 3) _____
Частка ТОП-3 у виторзі: _____ %
Рекомендації бізнесу: 1) _____
2) _____
3) _____

Критерії оцінювання (100 балів + 10 бонус)

Блок	Що перевіряється	Бали
Етапи 0-1	Базовий SQL: SELECT, WHERE, DISTINCT, агрегати, GROUP BY	15
Етапи 2-3	Дати, групування, віконні функції (LAG, SUM OVER), CTE	25
Етапи 4-5	JOIN-ланцюжки, HAVING, CASE, формулювання гіпотези	25
Етап 6	NULL / LEFT JOIN / NOT EXISTS / COALESCE, оцінка витоку	15
Етап 7	Ранжування, частки, концентрація виторгу	15
Звіт	Повнота й коректність висновків, рекомендації	5
Бонус 8-9	EXPLAIN ANALYZE, індекси, транзакції, ізоляція	+10

Додаток В. Шпаргалка корисних конструкцій

Без готових рішень — лише будівельні блоки, які знадобляться.

Навіщо	Конструкція
Місяць з дати	<code>date_trunc('month', order_date) • to_char(order_date, 'YYYY-MM') • EXTRACT(MONTH FROM order_date)</code>
Умовний підрахунок	<code>COUNT(*) FILTER (WHERE status='cancelled')</code> — рахує лише потрібні рядки
NULL-и	<code>COALESCE(x, 0) • col IS NULL • col IS NOT NULL</code>
Категоризація	<code>CASE WHEN rating<=2 THEN 'негатив' WHEN rating=3 THEN 'нейтрал' ELSE 'позитив' END</code>
Віконні функції	<code>LAG(v) OVER (ORDER BY m) • RANK() OVER (ORDER BY v DESC) • SUM(v) OVER (ORDER BY m)</code>
Частка від суми	<code>v * 100.0 / SUM(v) OVER ()</code>
Пошук «Відсутніх»	<code>LEFT JOIN ... WHERE r.id IS NULL або NOT EXISTS (SELECT 1 FROM ...)</code>
CTE	<code>WITH monthly AS (SELECT ...) SELECT ... FROM monthly</code>
Округлення	<code>ROUND(value::numeric, 2)</code>
План запиту	<code>EXPLAIN ANALYZE <запит>; • CREATE INDEX idx ON t(col);</code>

Додаток А. Повний DDL схеми

Цей код міститься у **setup.sql** (далі йдуть ще INSERT-и з даними). Наведено для довідки.

```
-- =====
-- Навчальна база даних "Інтернет-магазин" (PostgreSQL / OLTP)
-- Наскрізна схема для фінальної практики з SQL
-- =====

DROP TABLE IF EXISTS reviews, payments, order_items, orders,
                        products, employees, customers, suppliers, categories CASCADE;

-- Категорії товарів (самозв'язок: підкатегорії)
CREATE TABLE categories (
    category_id      SERIAL PRIMARY KEY,
    name             VARCHAR(60) NOT NULL,
    parent_category_id INT REFERENCES categories(category_id)
);

-- Постачальники
CREATE TABLE suppliers (
    supplier_id SERIAL PRIMARY KEY,
    name        VARCHAR(80) NOT NULL,
    country     VARCHAR(40),
    rating      NUMERIC(3,2)      -- 0.00 .. 5.00
);

-- Клієнти
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    full_name   VARCHAR(80) NOT NULL,
    email       VARCHAR(120) UNIQUE,
    city        VARCHAR(40),
    country     VARCHAR(40),
    created_at  DATE NOT NULL,
    is_active   BOOLEAN NOT NULL DEFAULT TRUE
);

-- Працівники (самозв'язок: manager_id -> employee_id)
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    full_name   VARCHAR(80) NOT NULL,
    position    VARCHAR(50),
    manager_id  INT REFERENCES employees(employee_id),
```

```

    hire_date    DATE NOT NULL,
    salary       NUMERIC(10,2),
    department   VARCHAR(40)
);

-- Товари
CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    category_id  INT REFERENCES categories(category_id),
    supplier_id  INT REFERENCES suppliers(supplier_id),
    price        NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    cost         NUMERIC(10,2) CHECK (cost >= 0),
    stock_quantity INT NOT NULL DEFAULT 0,
    created_at   DATE NOT NULL
);

-- Замовлення (employee_id може бути NULL -> онлайн-замовлення без менеджера)
CREATE TABLE orders (
    order_id     SERIAL PRIMARY KEY,
    customer_id  INT NOT NULL REFERENCES customers(customer_id),
    employee_id  INT REFERENCES employees(employee_id),
    order_date   DATE NOT NULL,
    status       VARCHAR(20) NOT NULL,    -- new / paid / shipped / delivered / cancelled
    shipping_city VARCHAR(40)
);

-- Позиції замовлення
CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id      INT NOT NULL REFERENCES orders(order_id),
    product_id    INT NOT NULL REFERENCES products(product_id),
    quantity      INT NOT NULL CHECK (quantity > 0),
    unit_price    NUMERIC(10,2) NOT NULL,
    discount      NUMERIC(4,2) NOT NULL DEFAULT 0    -- частка 0.00 .. 0.50
);

-- Оплати (не в кожного замовлення є оплата)
CREATE TABLE payments (
    payment_id   SERIAL PRIMARY KEY,
    order_id     INT NOT NULL REFERENCES orders(order_id),
    paid_at      TIMESTAMP,
    amount       NUMERIC(10,2) NOT NULL,
    method       VARCHAR(20),              -- card / cash / paypal
    status       VARCHAR(20)              -- success / pending / failed
);

-- Відгуки на товари
CREATE TABLE reviews (
    review_id    SERIAL PRIMARY KEY,
    product_id   INT NOT NULL REFERENCES products(product_id),
    customer_id  INT NOT NULL REFERENCES customers(customer_id),
    rating       INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment      TEXT,
    created_at   DATE NOT NULL
);

```